

Problem 1: The Last Astronaut (Pseudocode)

PROGRAM LastAstronautElimination

DEKLARASI:

N, K, posisi_sekarang, total_tereliminasi, hitungan, idx, nomor_dieliminasi :
INTEGER

astronot : ARRAY OF INTEGER // Menyimpan status keaktifan atau daftar astronot aktif

ALGORITMA:

INPUT N, K

// Inisialisasi daftar astronot aktif dari 1 sampai N

FOR i FROM 1 TO N DO

 astronot[i] = i

ENDFOR

posisi_sekarang = 1 // Dimulai dari astronot nomor 1

total_tereliminasi = 0

PRINT "Urutan astronot yang dieliminasi:"

// Perulangan hingga tersisa 1 astronot

WHILE total_tereliminasi < N - 1 DO

 hitungan = 1

 // Mencari posisi astronot ke-K yang masih aktif

 WHILE hitungan < K DO

```

posisi_sekarang = posisi_sekarang + 1

IF posisi_sekarang > N THEN
    posisi_sekarang = 1
ENDIF

// Hanya menghitung astronot yang belum dieliminasi
IF astronot[posisi_sekarang] != 0 THEN
    hitungan = hitungan + 1
ENDIF

ENDWHILE

// Proses eliminasi astronot pada posisi_sekarang
nomor_dieliminasi = astronot[posisi_sekarang]
PRINT nomor_dieliminasi
astronot[posisi_sekarang] = 0 // Tanda sudah dieliminasi
total_tereliminasi = total_tereliminasi + 1

// Memperbarui nilai K berdasarkan aturan nomor unik astronot
IF nomor_dieliminasi MOD 2 == 0 THEN
    K = K + 2
ELSE
    K = K - 1
ENDIF

// Validasi nilai batas minimum K
IF K < 2 THEN
    K = 2

```

```

ENDIF

// Memindahkan posisi awal hitungan ke astronot aktif berikutnya

REPEAT

    posisi_sekarang = posisi_sekarang + 1

    IF posisi_sekarang > N THEN

        posisi_sekarang = 1

    ENDIF

UNTIL astronot[posisi_sekarang] != 0 ATAU total_tereliminasi == N - 1

ENDWHILE

// Mencari dan menampilkan astronot terakhir yang bertahan

FOR i FROM 1 TO N DO

    IF astronot[i] != 0 THEN

        PRINT "Astronot terakhir yang bertahan: ", astronot[i]

    ENDIF

ENDFOR

END PROGRAM

```

Penjelasan Logika Problem 1

Algoritma di atas dirancang untuk menyelesaikan variasi dari masalah klasik Josephus Problem dengan aturan dinamis pergeseran nilai langkah (K). Struktur data utama menggunakan sebuah array berukuran (N) untuk merepresentasikan posisi melingkar para astronot di mana nilai elemen array yang diubah menjadi 0 menandakan bahwa astronot tersebut telah tereliminasi. Proses eliminasi dikendalikan oleh struktur perulangan bertingkat, di mana perulangan luar berjalan sampai jumlah astronot yang tereliminasi mencapai ($N-1$). Di dalam perulangan tersebut, program akan melakukan iterasi linear melingkar melewati elemen array yang bernilai bukan nol (masih aktif) untuk menghitung langkah hingga mencapai hitungan ke- K .

Ketika astronot ke- K ditemukan, identitas nomor unik astronot tersebut dievaluasi menggunakan operasi sisa bagi ($\text{MOD } 2$) guna menentukan perubahan nilai (K) selanjutnya. Jika nomor genap terdeteksi, nilai variabel (K) ditambah 2, sedangkan jika ganjil maka dikurangi 1 dengan batasan minimum bersyarat agar nilai (K) tidak pernah mengecil di bawah angka 2. Setelah status eliminasi dicatat dan dicetak, penunjuk posisi indeks array digeser secara sekuensial ke kanan mencari elemen aktif berikutnya sebagai titik awal perhitungan baru. Mekanisme ini terus berulang secara deterministik hingga menyisakan tepat satu elemen bukan nol yang kemudian diidentifikasi sebagai astronot terakhir yang berhasil bertahan dalam misi.

Problem 2: Alien-In-The-Middle (Pseudocode)

PROGRAM AlienCipherSystemWithSpaces

DEKLARASI:

```
pesan_asli, pesan_sandi : STRING
panjang, i, nilai_sekarang, nilai_sebelum, nilai_baru : INTEGER
char_sekarang, char_sebelum, char_baru : CHARACTER
```

ALGORITMA:

```
// Menggunakan GETLINE agar kalimat dengan spasi bisa terbaca utuh
GETLINE pesan_asli

panjang = HITUNG_PANJANG(pesan_asli)

IF panjang > 0 THEN
    // Huruf pertama tidak mengalami perubahan
    pesan_sandi = KONVERSI_KAPITAL(pesan_asli)

    // Memproses huruf kedua (indeks 1) hingga akhir teks
    FOR i FROM 1 TO panjang - 1 DO

        // JIKA karakter saat ini atau sebelumnya adalah spasi, jangan disandi
        IF pesan_asli[i] == ' ' OR pesan_asli[i-1] == ' ' THEN
            pesan_sandi = pesan_sandi + pesan_asli[i]
        ELSE
            char_sekarang = KONVERSI_KAPITAL(pesan_asli[i])
            char_sebelum = KONVERSI_KAPITAL(pesan_asli[i-1])

            // Transformasi karakter menjadi nilai alfabet (A=1, B=2, ..., Z=26)
            nilai_sekarang = ASCII_TO_INT(char_sekarang) - ASCII_TO_INT('A') + 1
            nilai_sebelum = ASCII_TO_INT(char_sebelum) - ASCII_TO_INT('A') + 1

            // Menghitung pergeseran berdasarkan huruf sebelumnya
```

```

        nilai_baru = nilai_sekarang + nilai_sebelum

        // Loop siklis jika pergeseran melewati batas huruf Z
        IF nilai_baru > 26 THEN
            nilai_baru = nilai_baru - 26
        ENDIF

        // Mengembalikan nilai integer menjadi karakter teks
        char_baru = INT_TO_ASCII(nilai_baru + ASCII_TO_INT('A') - 1)
        pesan_sandi = pesan_sandi + char_baru
    ENDIF
ENDFOR

PRINT "Pesan hasil sandi: ", pesan_sandi
ENDIF

END PROGRAM

```

Penjelasan Logika Problem 2

Logika penyelesaian masalah enkripsi sandi rahasia ini berfokus pada manipulasi array karakter (string) menggunakan transformasi nilai ASCII menjadi representasi posisi alfabet 1 hingga 26, dengan kemampuan tambahan untuk mengenali karakter spasi. Algoritma diawali dengan membaca seluruh baris kalimat masukan secara utuh menggunakan instruksi pembacaan khusus yang dapat mempertahankan spasi antarkata. Karakter pertama dari string masukan langsung dipertahankan tanpa memodifikasinya sama sekali sesuai aturan baku sistem sandi rahasia. Proses enkripsi sesungguhnya berjalan menggunakan skema perulangan indeks tunggal yang dimulai dari posisi indeks kedua hingga mencapai akhir panjang untaian karakter teks asli.

Pada setiap iterasi di dalam perulangan, algoritma menerapkan sistem seleksi kondisi berlapis untuk memeriksa keberadaan spasi. Jika karakter yang sedang diproses saat ini atau karakter teks asli tepat di sebelah kirinya merupakan sebuah spasi, algoritma secara otomatis akan melewati proses enkripsi matematis dan langsung menyalin karakter asli tersebut ke dalam variabel penampung. Namun, jika kedua karakter tersebut berupa huruf alfabet standar, program akan mengonversinya ke dalam nilai integer relatif terhadap huruf 'A'. Nilai alfabet huruf saat ini kemudian dijumlahkan dengan nilai alfabet dari huruf sebelum sandi diterapkan. Kondisi luapan alfabet yang melebihi angka 26 diatasi dengan operasi pengurangan senilai 26 agar hitungan kembali berputar secara siklis dari awal alfabet sebelum akhirnya dikonversi kembali menjadi karakter teks rahasia utuh.